ปลายภาค 1/69 · การเขียนโปรแกรมเชิงวัตถุ (Java)

OOP สำหรับคนที่ยังไม่เข้าใจ เริ่มจากศูนย์จนอ่านโค้ดสอบได้

ทุกเรื่องเริ่มจากเรื่องเปรียบเทียบในชีวิตจริง แล้วค่อยไปดูโค้ด
และอธิบายโค้ดทีละบรรทัดว่าแต่ละส่วนทำหน้าที่อะไร

0 พื้นฐานที่ต้องรู้ก่อน | 7 Array | 8 Inheritance | 9 Abstraction | 10 Polymorphism | 11 Exception | 12 GUI

0 · พื้นฐาน

Class กับ Object ต่างกันอย่างไร



Class = แบบแปลนบ้าน
กำหนดว่าบ้านทุกหลังต้องมีอะไร (มีกี่ห้อง สีอะไร) และทำอะไรได้ (เปิดประตู เปิดไฟ) แต่แบบแปลนยังอยู่อาศัยไม่ได้



Object = บ้านที่สร้างเสร็จจริง
สร้างจากแบบเดียวกันได้หลายหลัง แต่ละหลังมีค่าของตัวเอง เช่น หลังหนึ่งสีฟ้า อีกหลังสีชมพู



`new` = สั่งช่างสร้างบ้าน 1 หลังจากแบบ
ทุกครั้งที่เขียน `new` จะได้ object ใหม่เพิ่ม 1 ตัว

```
// แบบแปลน (class)
class Dog {
  String name;            // ข้อมูล: หมามีชื่อ
  void bark() {           // ความสามารถ: เห่าได้
    System.out.println(name + " โฮ่ง!");
  }
}

// สร้างของจริง (object) 2 ตัว
Dog a = new Dog();  a.name = "ถุงเงิน";
Dog b = new Dog();  b.name = "ถุงทอง";
a.bark();   // ถุงเงิน โฮ่ง!
b.bark();   // ถุงทอง โฮ่ง!
```

จุด `.` อ่านว่า **“ของ”** เช่น `a.name` คือชื่อ**ของ** a และ `a.bark()` คือสั่งให้ a เห่า

ส่วนประกอบของ class มี 3 อย่างเสมอ

```java
public class Student {
  private String name;        // ①
  private double gpa;         // ①
  public Student(String name, double gpa) { // ②
    this.name = name;
    this.gpa  = gpa;
  }
  public boolean isHonor() {   // ③
    return gpa >= 3.50;
  }
  public void showInfo() {     // ③
    System.out.println(name + " | GPA: " + gpa);
  }
}

Student s = new Student("สมชาย", 3.25); // เรียก ②
s.showInfo();      // สมชาย | GPA: 3.25
s.isHonor();       // false
```

1. **Attribute (ข้อมูล)** สิ่งที่ object “มี” เขียนเป็น `ชนิด ชื่อ;` และมักเป็น `private` (ซ่อนไว้ ห้ามคนนอกแก้)
2. **Constructor (ตัวสร้าง)** ชื่อ**เหมือนชื่อ class** และ**ไม่มี return type** ทำงานครั้งเดียวตอนสั่ง `new` เพื่อใส่ค่าเริ่มต้น
3. **Method (ความสามารถ)** สิ่งที่ object “ทำได้” โดยคำหน้าชื่อคือชนิดของค่าที่ส่งกลับ

คำ	ความหมาย
`this.name`	name **ของ object ตัวนี้** (ไว้แยกจากพารามิเตอร์ที่ชื่อซ้ำ)
`void`	ทำงานอย่างเดียว ไม่ส่งค่ากลับ
`boolean` / `double` …	ส่งค่าชนิดนั้นกลับด้วย `return`
`public` / `private`	ใครเรียกก็ได้ / เรียกได้แค่ใน class เดียวกัน
`static`	เป็นของ class ไม่ใช่ของ object เช่น `main`

ตัวแปร object คือ “รีโมท” ไม่ใช่ตัวของจริง



ตัวแปรชนิดพื้นฐาน (`int` , `double` , `boolean`) เก็บ**ค่าไว้ในกล่องโดยตรง**



ตัวแปร object (`String` , `Dog` , `Student`) เก็บแค่**รีโมท** ซึ่งชี้ไปหาของจริงที่อยู่ที่อื่นในหน่วยความจำ



`null` **= รีโมทที่ยังไม่ได้จับคู่กับทีวีเครื่องไหน** ถ้ากดปุ่มบนรีโมทนี้ จะได้ **NullPointerException**

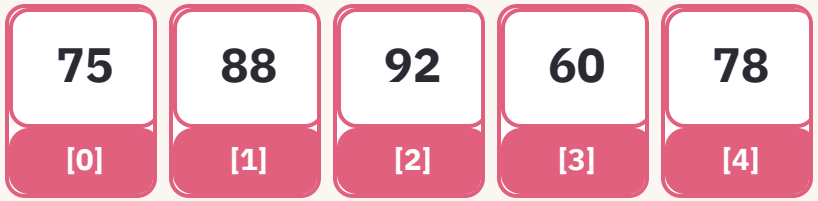
```java
int x = 5;                    // กล่อง x มีเลข 5 อยู่ข้างใน
Dog d;                        // มีรีโมทแล้ว แต่ยังไม่มีหมา
d = new Dog();                // สร้างหมา แล้วจับคู่กับรีโมท d
Dog e = d;                    // รีโมทอีกอัน ชี้หมาตัวเดียวกัน
e.name = "โบ้";
System.out.println(d.name); // โบ้  (เพราะเป็นหมาตัวเดียวกัน)

Dog z = null;
z.bark();                     // ✗ NullPointerException
```

เรื่องนี้เป็นพื้นฐานของบทที่ยากที่สุด (บท 10) ชนิดของรีโมทกำหนดว่า**มีปุ่มอะไรให้กด** ส่วนของจริงที่ถูกชี้กำหนดว่า**กดแล้วเกิดอะไรขึ้น**

Array = ตู้ล็อกเกอร์ที่ช่องแรกคือเลข 0

ถ้า `int[] scores = {75, 88, 92, 60, 78};` ตู้จะหน้าตาแบบนี้



- มี 5 ช่อง ช่องแรกคือ `[0]` และช่องสุดท้ายคือ `[4]` (**ไม่ใช่** [5])
- `scores.length` = 5 คือจำนวนช่อง (เขียน**ไม่มีวงเล็บ**)
- ตู้สร้างแล้ว**ขยายขนาดไม่ได้**
- เปิดช่อง [5] ซึ่งไม่มีอยู่จริง จะได้ **ArrayIndexOutOfBoundsException**
- ตู้ใหม่ที่ยังไม่ใส่อะไร: ถ้าเป็น `int` ทุกช่องเป็น 0 ถ้าเป็น object ทุกช่องเป็น `null`

```
int sum = 0;
for (int i = 0; i < scores.length; i++) {
  sum = sum + scores[i];
}
```

ลองไล่ลูปทีละรอบ (โจทย์ทายผลจะเป็นแบบนี้)

รอบ	i	scores[i]	sum หลังบวก
1	0	75	75
2	1	88	163
3	2	92	255
4	3	60	315
5	4	78	**393**
—	5	5 < 5 เป็นเท็จ จึงหยุดลูป	

บท 7 · Array 2 มิติ

Array 2 มิติ = ผังที่นั่งโรงหนัง [แถว][ที่นั่ง]

```
int[][] s = { {75, 80, 70},    // แถว 0
              {88, 70, 95},    // แถว 1
              {92, 85, 88} };  // แถว 2
```

	คอลัมน์ 0	คอลัมน์ 1	คอลัมน์ 2
แถว 0	75	80	70
แถว 1	88	70	**95**
แถว 2	92	85	88

`s[1][2]` = แถว 1 คอลัมน์ 2 = **95** · `s.length` = 3 แถว · `s[0].length` = 3 คอลัมน์

Array ของ object = ตู้ที่ในแต่ละช่องเก็บรีโมท

```
Student[] st = new Student[3];
// ได้ตู้ 3 ช่อง แต่ทุกช่องยังเป็น null (ยังไม่มีนักเรียน!)

st[0] = new Student("สมชาย", 3.25); // ต้อง new ทีละคน
st[0].showInfo();   // ✓
st[1].showInfo();   // × NullPointerException
```

ต้องจำ: `new Student[3]` สร้างแค่**ตู้** ยังไม่ได้สร้างนักเรียน ต้อง `new Student(...)` ใส่ทีละช่องเอง

บท 7 · ArrayList

ArrayList = แถวคิวที่ยืดหดได้เอง



มีคนมาต่อแถว คิวก็ยาวขึ้น มีคนออกจากแถว คนข้างหลังก็**ขยับขึ้นมาแทนเอง** ต่างจาก Array ที่ขนาดตายตัว

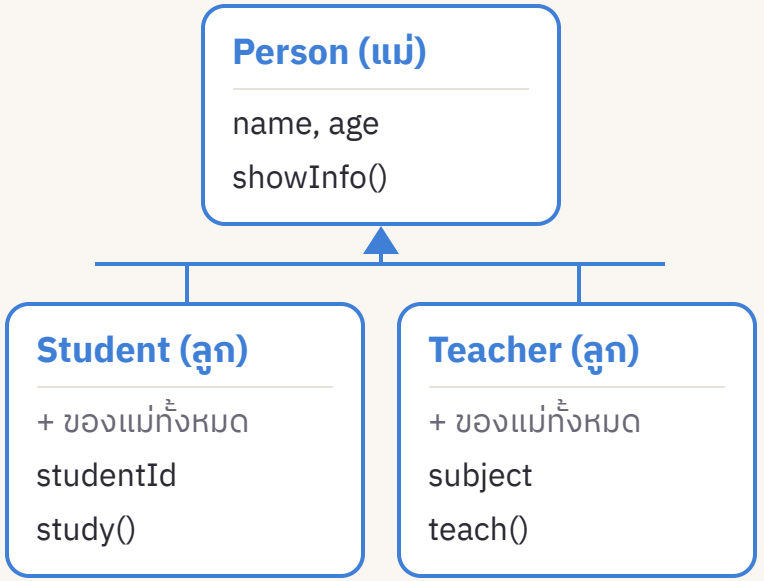
```
ArrayList<String> q = new ArrayList<>();
q.add("A");        // [A]
q.add("B");        // [A, B]
q.add("C");        // [A, B, C]
q.add(1, "X");     // แทรกที่ช่อง 1 → [A, X, B, C]
q.get(2);          // "B"
q.remove(0);       // [X, B, C]  ทุกคนขยับขึ้น
q.size();          // 3
```

คำสั่ง	ความหมายภาษาคน
`add(x)`	ต่อท้ายแถว
`add(i, x)`	แทรกเข้าไปที่ตำแหน่ง i
`get(i)`	ดูว่าตำแหน่ง i คือใคร
`set(i, x)`	เปลี่ยนคนที่ตำแหน่ง i
`remove(i)`	เอาคนที่ตำแหน่ง i ออก
`size()`	ตอนนี้มีกี่คน
`contains(x)`	มี x อยู่ในแถวไหม
`isEmpty()` / `clear()`	แถวว่างไหม / ไล่ทุกคนออก

`ArrayList<int>` **ผิด** ต้องเขียน `<Integer>` หรือ `<Double>` เพราะในวงเล็บแหลมต้องเป็นชื่อ class (ตัวพิมพ์ใหญ่)

Inheritance = ลูกได้รับมรดกจากพ่อแม่

ทุกอย่างที่แม่มี ลูกได้ไปด้วยอัตโนมัติ ไม่ต้องเขียนซ้ำ แล้วลูกยัง **เพิ่มของของตัวเอง**ได้




```
class Student extends Person {   // extends = "เป็นลูกของ"
  private String studentId;
  ...
}
```

ทดสอบว่าควรใช้ extends ไหม

พูดประโยค **"A เป็น B ชนิดหนึ่ง"** แล้วดูว่าฟังดูถูกไหม

- Student **เป็น** Person ชนิดหนึ่ง ✓ → ใช้ `extends` (**IS-A**)
- Car **เป็น** Engine ชนิดหนึ่ง × แต่ Car **มี** Engine → ให้ Engine เป็น attribute ของ Car (**HAS-A**)

ลูก**มีแม่ได้คนเดียว** (extends ได้ class เดียว) · ลูกใช้ของที่แม่ตั้งเป็น `private` โดยตรงไม่ได้ แต่ใช้ของที่เป็น `protected` และ `public` ได้

บท 8 · super

super = เรียกใช้ของพ่อแม่ และต้องสร้างพ่อแม่ก่อนเสมอ

```
class Person {
  String name;
  Person(String name) {
    this.name = name;
    System.out.println("① สร้างส่วน Person");
  }
}
class Student extends Person {
  String id;
  Student(String name, String id) {
    super(name);        // ส่ง name ไปให้ constructor แม่
    this.id = id;
    System.out.println("② สร้างส่วน Student");
  }
}
new Student("Alice", "65001");
// ① สร้างส่วน Person
// ② สร้างส่วน Student
```

🏗️ สร้างบ้านต้อง**ลงฐานราก (แม่) ก่อน** แล้วค่อยสร้างชั้นบน (ลูก) ผลลัพธ์จึงขึ้นของแม่ก่อนเสมอ

เขียน	หมายถึง
`super(name)`	เรียก constructor ของแม่ ต้องอยู่**บรรทัดแรก**ของ constructor ลูก
`super.showInfo()`	เรียกเมธอดเวอร์ชันของแม่
`this.id`	attribute ของตัวเอง

ถ้าไม่ได้เขียน `super(...)` เลย Java จะแอบใส่ `super()` (แบบไม่มีพารามิเตอร์) ให้ ถ้าแม่**ไม่มี** constructor แบบนั้น จะ **compile error**

บท 8 · Override

Override = ลูกทำสิ่งเดียวกันในแบบของตัวเอง

🗣️ แม่สอนว่า “ทักทาย” คือพูดว่า *สวัสดี* แต่ลูกที่ไปเรียนต่างประเทศ ทักทายด้วย *Hello* แทน ชื่อการกระทำเหมือนเดิม แต่**วิธีทำ**เปลี่ยนไป

```
class Animal {
  void makeSound() { System.out.println("..."); }
}
class Dog extends Animal {
  @Override                              // บอกว่าตั้งใจเขียนทับ
  void makeSound() { System.out.println("โฮ่ง"); }
}
new Dog().makeSound();    // โฮ่ง (ใช้ของลูก)
```

เงื่อนไขของ Override

- ชื่อเมธอดและวงเล็บพารามิเตอร์**ต้องเหมือนแม่ทุกตัวอักษร**
- ระดับการเข้าถึงต้องเท่าเดิมหรือ**กว้างกว่า** เช่น แม่เป็น public ลูกจะเป็น private ไม่ได้
- เมธอด `private`, `final`, `static` และ constructor **override ไม่ได้**

Overriding (บท 8, 10)	Overloading (ชื่อคล้าย อย่าสับสน)
อยู่ใน**แม่กับลูก**	อยู่ใน**class เดียวกัน**
พารามิเตอร์**เหมือน**กัน	พารามิเตอร์**ต่าง**กัน เช่น add(int,int) กับ add(double,double)
Java เลือกตอน**รัน**	Java เลือกตอน **compile**

บท 9 · Abstract class

Abstract class = แม่พิมพ์ที่ยังไม่เสร็จ

🔷 ถ้ามีคนบอกว่า “วาด**รูปทรง**มาหนึ่งรูป” คุณจะถามกลับว่ารูปทรงอะไร เพราะ “รูปทรง” เป็นแค่**แนวคิด** ส่วนวงกลมหรือสี่เหลี่ยมถึงจะวาดได้จริง

- รูปทรงทุกแบบ**ต้อง**หาพื้นที่ได้ แต่สูตรต่างกัน แม่จึงประกาศไว้แค่ชื่อเมธอด (**abstract method** ซึ่งไม่มีเนื้อ ไม่มี {})
- ลูกทุกตัว**ถูกบังคับ**ให้เขียนเนื้อเมธอดนั้น
- สั่ง `new Shape()` **ไม่ได้** เพราะแม่พิมพ์ยังไม่สมบูรณ์

```
abstract class Shape {
  abstract double calculateArea();  // ไม่มี { } ปิดด้วย ;
}
class Circle extends Shape {
  double radius;
  Circle(double r) { radius = r; }
  @Override
  double calculateArea() { return Math.PI * radius * radius; }
}
class Rectangle extends Shape {
  double w, h;
  Rectangle(double w, double h) { this.w = w; this.h = h; }
  @Override
  double calculateArea() { return w * h; }
}
Shape s1 = new Shape();      // ✗ ห้าม
Shape s2 = new Circle(5);    // ✓ รีโมทแม่ ชี้ของจริงที่เป็นลูก
```

ใน UML ชื่อ class ที่เป็น**ตัวเอียง** หรือมีคำว่า **«abstract»** กำกับ แปลว่าต้องเขียน `abstract`

บท 9 · Interface

Interface = สัญญาที่บอกว่า “ต้องทำอะไรได้บ้าง”

🔌 เต้าเสียบไฟกำหนดแค่**รูปทรงหัวปลั๊ก** ไม่สนใจว่าเครื่องที่เสียบจะเป็นพัดลมหรือทีวี ขอแค่หัวปลั๊กตรงตามรูป ก็เสียบได้

```
interface PaymentMethod {          // สัญญา
  void pay(double amount);         // ต้องจ่ายเงินได้
}
class CashPayment implements PaymentMethod {
  public void pay(double amount) {    // ต้องมี public
    System.out.println("จ่ายเงินสด " + amount);
  }
}
class QRPayment implements PaymentMethod {
  public void pay(double amount) {
    System.out.println("สแกนจ่าย " + amount);
  }
}
```

	abstract class	interface
เปรียบเหมือน	แม่พิมพ์ที่ยังไม่เสร็จ	สัญญา / ใบรับรอง
ลูกใช้คำว่า	`extends`	`implements`
ใช้ได้กี่อัน	แค่ **1**	**หลายอัน** คั่นด้วย ,
มีข้อมูล (attribute)?	มีได้	มีได้แค่ค่าคงที่
มี constructor?	มี	ไม่มี
สั่ง new ได้?	ไม่ได้	ไม่ได้

```
class Phone implements Camera, MusicPlayer {}  // ได้หลายอัน
class FullTime extends Employee implements Taxable {}
// extends ต้องมาก่อน implements
```

เมธอดที่ implement จาก interface ต้องเขียน `public` เสมอ ถ้าลืมจะ compile error

บท 10 · Polymorphism

Polymorphism = สั่งคำเดียว แต่ละตัวทำไม่เหมือนกัน

📢 ครูตะโกนว่า “**ทุกตัวส่งเสียง!**” หมาเห่า แมวร้องเหมียว วัวร้องมอ ครูไม่ต้องรู้ว่าใครเป็นสัตว์อะไร แค่สั่งคำเดียว

```
Animal[] zoo = { new Dog(), new Cat(), new Cow() };
for (Animal a : zoo) {
  a.makeSound();      // คำสั่งเดียวกันทุกตัว
}
// Dog barks: Woof!
// Cat meows: Meow!
// Cow moos: Moo!
```

ถ้าเพิ่มเป็ดเข้ามา แค่สร้าง `class Duck extends Animal` แล้วใส่ลง array **โดยไม่ต้องแก้ลูปเลย**

กฎรีโมท (ย้อนไปดูหน้า 4)

```
Animal a = new Dog();
//  ↑ รีโมทแบบ Animal    ↑ ของจริงเป็น Dog
```

- **กดปุ่มอะไรได้** ดูจากรีโมท (Animal) ถ้า Animal ไม่มีปุ่ม `guardHouse()` ก็ **compile error** ถึงแม้หมาจะเฝ้าบ้านได้
- **กดแล้วเกิดอะไร** ดูจากของจริง (Dog) `a.makeSound()` จึงได้เสียงหมา

ข้อยกเว้นที่ชอบออกสอบ: attribute (ตัวแปรใน class) **ไม่ใช้กฎนี้** ถ้าแม่กับลูกมี `name` ทั้งคู่ `a.name` จะได้ค่าของ**รีโมท (แม่)**

บท 10 · Casting

เปลี่ยนรีโมท: Upcasting และ Downcasting

	Upcasting ⬆️	Downcasting ⬇️
ทิศทาง	ลูก → แม่	แม่ → ลูก
เขียนยังไง	`Animal a = dog;` ทำให้อัตโนมัติ	`Dog d = (Dog) a;` ต้องเขียน (ชนิด) เอง
ปลอดภัย?	ปลอดภัยเสมอ	ถ้าของจริงไม่ใช่ชนิดนั้น จะพังตอนรัน

🐱 เอารีโมท “สัตว์” ที่จริง ๆ ชี้ไปหา**แมว** แล้วไปบังคับว่า “นี่คือหมา” Java ยอมให้ compile ผ่าน แต่ตอนรันจะเจอ **ClassCastException**

```
Animal a = new Dog();
Dog d = (Dog) a;          // ✓ ของจริงเป็นหมา
d.guardHouse();           // ✓ ตอนนี้มีปุ่มนี้แล้ว

Animal x = new Cat();
Dog y = (Dog) x;          // ✗ ClassCastException

// วิธีปลอดภัย: เช็คด้วย instanceof ก่อนแปลง
if (x instanceof Dog) {
  Dog z = (Dog) x;
  z.guardHouse();
}
```

`x instanceof Dog` อ่านว่า “x เป็นหมาจริงไหม” ได้ผลเป็น `true` หรือ `false`

สูตร 4 ขั้นสำหรับข้อ “ผลลัพธ์ของโปรแกรมคือ?”

1. **เจอ `new` ให้ดูว่าเป็น class อะไร** ถ้าเป็นลูก ให้รัน constructor ของ **แม่ก่อน** แล้วค่อยรันของลูก
2. **เจอการเรียกเมธอด ให้ถามว่ารีโมทมีปุ่มนี้ไหม** ถ้าไม่มี ให้ตอบว่า compile error ทันที
3. **ถ้ามีปุ่ม ให้ไปดูของจริง** ถ้าของจริงเป็นลูกที่ override เมธอดนี้ไว้ ให้ใช้เวอร์ชันลูก ถ้าไม่ได้ override ให้ใช้ของแม่
4. **เจอ `super.xxx()` ข้างในเมธอดลูก** ให้รันเวอร์ชันของแม่ตรงนั้น แล้วกลับมาทำบรรทัดต่อไปของลูก

```
class A {
  A() { System.out.print("A "); }
  void hi() { System.out.print("hiA "); }
}
class B extends A {
  B() { System.out.print("B "); }
  void hi() { super.hi(); System.out.print("hiB "); }
}
A obj = new B();    // ขั้น 1 → A B
obj.hi();           // ขั้น 2: A มีปุ่ม hi ✓
                    // ขั้น 3: ของจริงคือ B → hi ของ B
                    // ขั้น 4: super.hi() → hiA แล้วต่อ hiB
```

คำตอบ

```
A B hiA hiB
```

Exception = ตาข่ายนิรภัยกันโปรแกรมพัง



นักกายกรรมเล่นบนเชือก (**try**) ถ้าตกลงมา ก็มีตาข่าย (**catch**) รองรับไว้ แล้วไม่ว่าจะตกหรือไม่ จบการแสดงก็ต้องโค้งคำนับเสมอ (**finally**)

```
try {
  // โค้ดที่อาจจะพัง
  int r = 10 / 0;
  System.out.println("ไม่ถูกพิมพ์");
} catch (ArithmeticException e) {
  // ทำอะไรเมื่อพังแบบนี้
  System.out.println("หารศูนย์ไม่ได้");
} finally {
  System.out.println("จบ");        // ทำเสมอ
}
// หารศูนย์ไม่ได้
// จบ
```

try รันไปทีละบรรทัด → **พัง?** ข้ามบรรทัดที่เหลือใน try ทันที → **catch** เลือกตัว **แรก** ที่ชนิดตรงกัน → **finally** ทำเสมอ

ชื่อ Exception	เกิดเมื่อ
ArithmeticException	หาร int ด้วย 0
ArrayIndexOutOfBoundsException	เปิดช่อง array ที่ไม่มีอยู่
NumberFormatException	แปลงข้อความที่ไม่ใช่ตัวเลข เช่น `Integer.parseInt("ABC")`
NullPointerException	ใช้รีโมทที่เป็น null
InputMismatchException	ใช้ Scanner รอรับตัวเลข แต่ผู้ใช้พิมพ์ตัวอักษร
ClassCastException	downcast ผิดชนิด (บท 10)

ชื่อ Exception บอกสาเหตุอยู่แล้ว ถ้าอ่านคำแรกของชื่อออก ก็เดาได้ว่าเกิดจากอะไร

throw คือ “โยน” ส่วน throws คือ “ป้ายเตือน”

🚨 **throw** = เห็นข้อมูลผิดแล้วกดสัญญาณเตือนเอง ณ บรรทัดนั้น
throws = ป้ายหน้าห้องที่บอกว่า “ห้องนี้อาจมีสัญญาณดัง คนเรียกใช้เตรียมรับมือด้วย”

```
public void setAge(int age) {
  if (age < 0) {
    throw new IllegalArgumentException("Age cannot be negative");
  }
  this.age = age;
}

public void readFile() throws IOException { ... }
```

สร้าง Exception ของตัวเอง (โครงที่ตายตัว)

```
class InsufficientBalanceException extends Exception {
  InsufficientBalanceException(String msg) {
    super(msg);
  }
}
```

Unchecked	Checked
compiler **ไม่บังคับ**ให้ดัก	compiler **บังคับ**ให้ try-catch หรือเขียน throws
5 ตัวในหน้าที่แล้วทั้งหมด	FileNotFoundException, IOException (เรื่องไฟล์)

ถ้ามีหลาย catch ให้เรียง**ตัวเจาะจงไว้ก่อน** แล้ว `catch (Exception e)` ไว้**ท้ายสุด** ถ้าสลับกันจะ compile error

บท 12 · GUI

GUI = จัดร้านอาหาร

ในร้าน	ใน Java Swing
ตัวร้าน	**JFrame** หน้าต่างหลัก
โต๊ะแต่ละตัว	**JPanel** ใช้จัดกลุ่มของ
ป้ายเมนู	**JLabel** ข้อความที่ผู้ใช้แก้ไม่ได้
กระดาษให้ลูกค้าเขียน	**JTextField** 1 บรรทัด / **JTextArea** หลายบรรทัด
กริ่งเรียกพนักงาน	**JButton**
เลือกท็อปปิงได้หลายอย่าง	**JCheckBox**
เลือกขนาดได้อย่างเดียว	**JRadioButton** + ButtonGroup
รายการให้เลือก	**JComboBox** (dropdown)
ป๊อปอัปแจ้งลูกค้า	**JOptionPane**
วิธีจัดโต๊ะในร้าน	**Layout Manager**

BorderLayout แบ่ง 5 ทิศ



- **FlowLayout** เรียงจากซ้ายไปขวาเหมือนตัวหนังสือ เต็มแล้วขึ้นบรรทัดใหม่
- **GridLayout(3, 2)** ตาราง 3 แถว 2 คอลัมน์ ทุกช่องขนาดเท่ากัน
- ฟอร์มจริงมักเอา JPanel ที่เป็น Grid ไปใส่ใน**ทิศ**ของ BorderLayout อีกที

โปรแกรม GUI ทุกตัวมีโครงเดียวกัน

```
import javax.swing.*;  import java.awt.*;
public class HelloGUI extends JFrame {     // ①
  private JTextField nameField;            // ②
  private JLabel resultLabel;

  public HelloGUI() {
    nameField = new JTextField(15);
    JButton btn = new JButton("ทักทาย");
    resultLabel = new JLabel("กรุณากรอกชื่อ");
    setLayout(new FlowLayout());           // ③
    add(nameField); add(btn); add(resultLabel);
    btn.addActionListener(e -> greet());   // ④
    setTitle("Hello"); setSize(400, 150);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
  }
  private void greet() {                   // ⑤
    String name = nameField.getText().trim();
    if (name.isEmpty()) resultLabel.setText("กรุณากรอกชื่อ");
    else resultLabel.setText("สวัสดี " + name);
  }
  public static void main(String[] args) {
    SwingUtilities.invokeLater(() ->
        new HelloGUI().setVisible(true));    // ⑥
  }
}
```

1. หน้าต่างของเรา**เป็น JFrame ชนิดหนึ่ง** ซึ่งก็คือ inheritance จากบท 8
2. ประกาศช่องกรอกและป้ายไว้**ด้านบน** เพื่อให้เมธอดอื่นเรียกใช้ได้
3. เลือกวิธีจัดวางก่อน แล้วค่อย `add` ของลงหน้าต่าง
4. **ผูกปุ่มกับงาน** คือเมื่อคลิกปุ่ม ให้ไปทำ `greet()` ถ้าไม่มีบรรทัดนี้ ปุ่มจะกดแล้วไม่เกิดอะไร
5. `getText()` อ่านสิ่งที่ผู้ใช้พิมพ์ · `trim()` ตัดช่องว่างหัวท้าย · `setText()` เปลี่ยนข้อความที่แสดง
6. สร้างหน้าต่างแล้วสั่ง**แสดงผล** ถ้าลืม `setVisible(true)` จะไม่เห็นอะไรขึ้นมาเลย

บท 11 + 12 · ข้อเติมโค้ด

ช่องตัวเลขใน GUI ต้องกันผู้ใช้พิมพ์ผิดเสมอ

```
private void calculate() {
  try {
    double a = Double.parseDouble(firstField.getText().trim());
    double b = Double.parseDouble(secondField.getText().trim());
    resultLabel.setText("ผลลัพธ์ = " + (a + b));
  } catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(this,
        "กรุณากรอกข้อมูลเป็นตัวเลข",
        "ข้อมูลไม่ถูกต้อง",
        JOptionPane.ERROR_MESSAGE);
  }
}
```

ทุกอย่างที่ได้จาก `getText()` เป็น**ข้อความ** (String) ถ้าจะเอาไปคำนวณ ต้องแปลงด้วย `parseDouble` หรือ `parseInt` ก่อน และถ้าผู้ใช้พิมพ์ “abc” จะเกิด **NumberFormatException**

กล่องป๊อปอัป JOptionPane	ใช้ทำอะไร / ได้อะไรกลับมา
`showMessageDialog`	แจ้งข้อความ แล้วมีปุ่ม OK
`showInputDialog`	ให้พิมพ์ ได้ String กลับมา หรือได้ **null** ถ้ากด Cancel
`showConfirmDialog`	ถาม Yes/No ได้ int กลับมา ให้เทียบกับ `YES_OPTION`

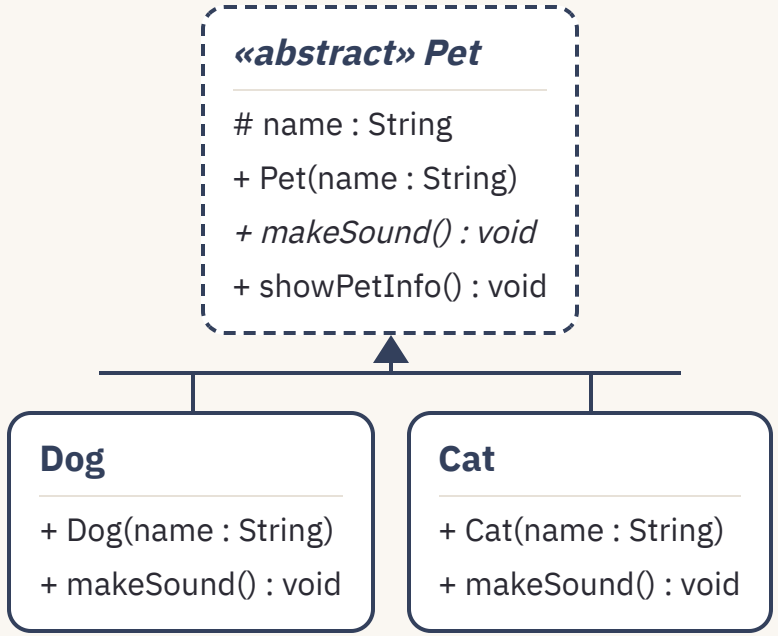
ไอคอนของป๊อปอัป

`INFORMATION_MESSAGE` ข้อมูล · `WARNING_MESSAGE` เตือน · `ERROR_MESSAGE` ผิดพลาด · `QUESTION_MESSAGE` คำถาม · `PLAIN_MESSAGE` ไม่มีไอคอน

ข้อเติมโค้ดมักเว้นว่างตรง: `try` · `catch` · `NumberFormatException` · `parseDouble` · `getText` · `setText` · `addActionListener` · `showMessageDialog` · `setVisible`

ตอนที่ 2.1 · UML → โค้ด

อ่าน UML Class Diagram ให้เป็นโค้ดทีละขั้น



สัญลักษณ์	แปลว่า
+ # −	public · protected · private
`name : String`	เขียนกลับด้านเป็น `String name;`
ตัวเอียง / «abstract»	ใส่คำว่า `abstract`
ลูกศรหัวสามเหลี่ยมเส้นทึบ / เส้นประ	`extends` / `implements`

```java
public abstract class Pet {
  protected String name;
  public Pet(String name) { this.name = name; }
  public abstract void makeSound();
  public void showPetInfo() {
    System.out.print(name + " Making Sound : ");
    makeSound();
  }
}
public class Dog extends Pet {
  public Dog(String name) { super(name); }
  @Override
  public void makeSound() {
    System.out.println("Bok Bok...");
  }
}
```

1. เขียนแม่ก่อน: attribute → constructor → เมธอด
2. ลูกทุกตัว: `extends` + constructor ที่มี `super(...)` บรรทัดแรก
3. ลูกเขียนเนื้อ abstract method ให้ครบ พร้อม `@Override`

ทวน

พจนานุกรมคำสำคัญ เห็นคำนี้ในโค้ด ให้นึกถึงความหมายนี้

คำ	นึกถึง
`class`	แบบแปลน
`new`	สร้างของจริง 1 ชิ้น
`this`	ตัวฉันเอง
`extends`	เป็นลูกของ (IS-A)
`super`	พ่อแม่ของฉัน
`@Override`	ฉันทำเรื่องนี้ในแบบของฉัน
`abstract`	ยังไม่เสร็จ ลูกต้องทำต่อ
`interface` / `implements`	สัญญา / ลงชื่อรับสัญญา
`instanceof`	ของจริงเป็นชนิดนี้ไหม

คำ	นึกถึง
`private` / `protected` / `public`	เฉพาะฉัน / ครอบครัว / ทุกคน
`static`	ของส่วนกลาง (เป็นของ class)
`final`	ห้ามแก้ ห้าม override
`null`	รีโมทที่ไม่ได้ชี้อะไร
`try` / `catch` / `finally`	ลองทำ / ตาข่าย / ทำเสมอ
`throw` / `throws`	กดสัญญาณเตือน / ป้ายเตือน
`ArrayList<T>`	แถวคิวที่ยืดหดได้
`addActionListener`	ผูกปุ่มกับงาน
`getText` / `setText`	อ่าน / เขียนข้อความบนหน้าจอ

เสาหลักที่ 4 · Encapsulation

Encapsulation = ล็อกข้อมูลไว้ แล้วเปิดให้ใช้ผ่านประตูที่มีคนตรวจ



เงินในธนาคารอยู่ในตู้เซฟ (**private**) ลูกค้าเปิดตู้เองไม่ได้ ต้องผ่าน**พนักงาน** (**getter/setter**) ซึ่งจะตรวจก่อนเสมอ เช่น ห้ามฝากเงินติดลบ

```
class Account {
  private double balance;          // ① ซ่อนข้อมูล

  public double getBalance() {     // ② getter = อ่าน
    return balance;
  }
  public void deposit(double amount) { // ③ setter = เขียน
    if (amount > 0) balance += amount;  // ตรวจก่อน
  }
}
acc.balance = -500;   // ✗ compile error (private)
acc.deposit(500);     // ✓
```

4 เสาหลัก OOP	จำสั้น ๆ	เห็นในโค้ดเป็น
Encapsulation	ซ่อนข้อมูล	`private` + getter/setter
Inheritance	รับมรดก	`extends`, `super`
Polymorphism	สั่งเหมือน ทำต่าง	`Animal a = new Dog()`, override, overload
Abstraction	บอกแค่ “ทำอะไร”	`abstract`, `interface`

ชอบถามสลับ: Encapsulation = **ปกป้องข้อมูล** (ซ่อน “ข้อมูล”) · Abstraction = **ซ่อนวิธีทำ** (ซ่อน “ความซับซ้อน”)

getter ชื่อขึ้นต้น `get…()` · ถ้าคืน boolean มักขึ้นต้น `is…()` เช่น `isHonor()` · setter ขึ้นต้น `set…(x)`

ออกสอบแน่ · Overload vs Override

Overload กับ Override ต่างกันยังไง (เทียบโค้ดข้างกัน)

Overloading: ชื่อเดียว · พารามิเตอร์ต่าง · class เดียวกัน

```
class Calculator {
  int    add(int a, int b)          { return a + b; }
  double add(double a, double b)    { return a + b; }
  int    add(int a, int b, int c)   { return a + b + c; }
}
c.add(10, 20);       // 30
c.add(5.5, 2.5);     // 8.0
c.add(10, 20, 30);   // 60
```

Java เลือกเมธอดจาก**จำนวน/ชนิดของค่าที่ส่ง** ตั้งแต่ตอน compile

Overriding: ชื่อเดียว · พารามิเตอร์เหมือน · แม่กับลูก

```
class Animal { void makeSound() { System.out.println("..."); } }
class Dog extends Animal {
  @Override void makeSound() { System.out.println("Woof"); }
}
Animal a = new Dog();  a.makeSound();   // Woof
```

	Overloading	Overriding
อยู่ที่ไหน	class **เดียวกัน**	**แม่กับลูก** (ต้องมี extends/implements)
พารามิเตอร์	**ต่าง** (จำนวน/ชนิด/ลำดับ)	**เหมือนทุกตัว**
return type	ต่างได้	ต้องเหมือน
ตัดสินตอน	**Compile time**	**Runtime**
ชื่อเรียก	compile-time polymorphism	runtime polymorphism
ทำไปเพื่อ	รับข้อมูลได้หลายแบบ	ให้ลูกเปลี่ยนพฤติกรรม
ใช้ @Override	ไม่ใช้	ใช้

กับดัก: ถ้าต่างกัน**แค่ return type** (พารามิเตอร์เหมือนกัน) = **compile error** ไม่นับเป็น overloading · constructor ก็ overload ได้ แต่ override ไม่ได้

ออกสอบ · ต้อง import อะไร

import แบบเต็ม (เหมือนช้อยส์ในข้อสอบ)

java.util

```
import java.util.ArrayList;
import java.util.Scanner;
import java.util.InputMismatchException;
```

java.awt (Layout / สี / ฟอนต์)

```
import java.awt.FlowLayout;
import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.Color;
import java.awt.Font;
```

java.awt.event (Event)

```
import java.awt.event.ActionListener;
import java.awt.event.ActionEvent;
```

javax.swing (ทุกตัวที่ขึ้นต้นด้วย J)

```
import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JLabel;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JCheckBox;
import javax.swing.JRadioButton;
import javax.swing.ButtonGroup;
import javax.swing.JComboBox;
import javax.swing.JOptionPane;
import javax.swing.SwingUtilities;
```

javax.swing.table · java.io

```
import javax.swing.table.DefaultTableModel;
import java.io.IOException;
```

ไม่ต้อง import เลย

String · Math · System · Integer · Double · Exception · ArithmeticException · NullPointerException · NumberFormatException · ArrayIndexOutOfBoundsException

ช้อยส์หลอกที่ชอบเจอ

ActionListener อยู่ **java.awt.event**
ไม่ใช่ javax.swing
JOptionPane อยู่ **javax.swing**
ไม่ใช่ java.awt
BorderLayout อยู่ **java.awt**
ไม่ใช่ javax.swing
Scanner / ArrayList อยู่ **java.util**
ไม่ใช่ java.io / java.lang

ออกสอบ · โค้ดนี้ใช้หลักการอะไร

เห็นโค้ดแบบนี้ → ตอบหลักการนี้

เห็นโค้ด	ตอบ
`class Dog extends Animal`	**Inheritance** (IS-A)
`private` + `getX()` / `setX()`	**Encapsulation**
`abstract class` , `abstract void m();`	**Abstraction**
`class Cash implements PaymentMethod`	**Interface** (Abstraction)
class เดียวมี `add(int,int)` และ `add(double,double)`	**Overloading** (compile-time poly)
ลูกเขียนเมธอดชื่อ+พารามิเตอร์เหมือนแม่ / `@Override`	**Overriding** (runtime poly)
`Animal a = new Dog();`	**Polymorphism** / Upcasting
`Dog d = (Dog) a;`	**Downcasting**

เห็นโค้ด	ตอบ
`for (Animal a : zoo) a.makeSound();`	**Polymorphism** (dynamic binding)
`super(name);`	เรียก **constructor แม่**
`this.name = name;`	อ้าง field ของ **object ตัวเอง**
class มี field เป็น object อื่น `private Engine engine;`	**Composition** (HAS-A)
`try { } catch (…) { } finally { }`	**Exception handling**
`class MyEx extends Exception`	**Custom exception**
`btn.addActionListener(e -> …)`	**Event handling** + Lambda
`ArrayList<Integer>` + `list.add(5)`	**Generics** + Autoboxing

ออกสอบ · ต่างกันยังไง

คู่ที่ชอบถามว่า “ต่างกันยังไง” ตอบสั้น 1 บรรทัด

คู่	ต่างกัน
Overload / Override	class เดียว พารามิเตอร์ต่าง / แม่-ลูก พารามิเตอร์เหมือน
extends / implements	สืบ class ได้ 1 / ทำตาม interface ได้หลายตัว
abstract class / interface	มี constructor+field / เป็นแค่สัญญา มีแต่ค่าคงที่
this / super	ตัวเอง / พ่อแม่
IS-A / HAS-A	extends / เก็บเป็น field (composition)
Upcast / Downcast	ลูก → แม่ อัตโนมัติ / แม่ → ลูก ต้อง cast เสี่ยงพัง
Encapsulation / Abstraction	ซ่อนข้อมูล / ซ่อนวิธีทำ
Array / ArrayList	ขนาดคงที่ `.length` / ยืดหด `.size()`

คู่	ต่างกัน
throw / throws	โยนในเนื้อเมธอด / ประกาศที่หัวเมธอด
final / finally	ห้ามแก้/ห้าม override / บล็อกที่ทำเสมอ
Checked / Unchecked	compiler บังคับ (IOException) / ไม่บังคับ (RuntimeException)
Compile / Runtime error	ผิดไวยากรณ์ รันไม่ได้ / รันแล้วพัง
== / equals()	เทียบที่อยู่ / เทียบเนื้อหา (String ใช้ equals)
JCheckBox / JRadioButton	เลือกหลายข้อ / ข้อเดียว (+ButtonGroup)
JTextField / JTextArea	1 บรรทัด / หลายบรรทัด
AWT / Swing	รุ่นเก่า java.awt / รุ่นใหม่ javax.swing ขึ้นต้น J

กลยุทธ์สำหรับมือใหม่

เก็บคะแนนให้ได้มากที่สุดแม้ยังไม่เข้าใจทั้งหมด

ตอน 1 ปรนัย

- ข้อที่ถามว่า “compile error ไหม” ให้มองหา `new` abstract หรือ interface, เรียกปุ่มที่รีโมทไม่มี, ลืม `super`, catch ผิดลำดับ
- ข้อ array: ช่องสุดท้ายคือ length−1
- ข้อ try: บรรทัดหลังจุดที่พังไม่ถูกรัน และ finally ทำเสมอ
- ไล่ค่าตัวแปรด้วยการจดลงกระดาษทุกครั้ง

ตอน 2.1 UML

- คะแนนให้ตามส่วน ถึงทำไม่ครบก็เขียน **โครง**ให้ครบไว้ก่อน
- เขียน `class X extends Y`, attribute และ constructor พร้อม `super` ให้ถูก ก็ได้คะแนนไปเกินครึ่ง
- main: ใช้ `ArrayList<แม่>` + `add(new ลูก(...))` + for-each เรียกเมธอด

ตอน 2.2 เติมโค้ด

- ดูคำที่อยู่**รอบ ๆ ช่องว่าง** ถ้าเห็นเปิดปีกกาตามด้วย `catch` ช่องข้างบนคือ `try`
- เห็น `parseInt` หรือ `parseDouble` ให้ catch `NumberFormatException`
- เห็นการหาร ให้ catch `ArithmeticException`
- เห็นปุ่ม ให้ใช้ `addActionListener(e -> ...)`
- บรรทัดสุดท้ายของ main มักเป็น `setVisible(true)`

ฝึกข้อเขียน

ฝึกเขียนโค้ดตอนที่ 2: 8 ข้อ พร้อมเฉลย

วิธีใช้ชุดนี้

1. อ่านโจทย์ 1 หน้า แล้ว**ปิดเฉลย** เขียนลงกระดาษ 10–15 นาที
2. เปิดหน้าถัดไปเป็นเฉลย ตรวจทีละบรรทัด
3. จด “จุดที่ผิด” ไว้ แล้วกลับไปเขียนข้อนั้นใหม่อีกรอบ

ถ้าเวลาน้อย ทำ **ข้อ 4 (Pet)** กับ **ข้อ 7 (GUI บวกเลข)** ก่อน เพราะตรงกับสไลด์อาจารย์ที่สุด

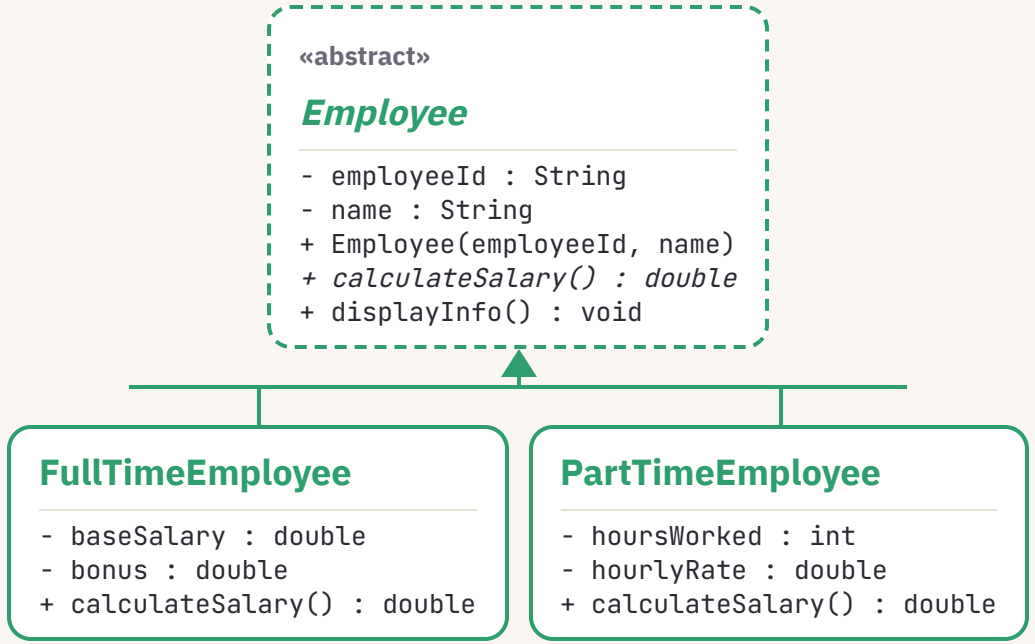
เขียนไม่ครบก็ได้คะแนน

1. **หัวคลาส** `abstract class / extends / implements`
2. **attribute** ตาม UML (– private, # protected, + public)
3. **constructor** + `super(...)` บรรทัดแรก
4. **abstract method** ในแม่ + `@Override` ในลูก
5. **main**: `ArrayList<แม่>` + for-each

ลืม syntax อย่าเว้นว่าง เขียนคอมเมนต์อธิบายแทนก็ยังได้คะแนนความเข้าใจ

ฝึกข้อเขียน · ข้อ 1 (2.1 UML → โค้ด)

ข้อ 1 · ระบบเงินเดือนพนักงาน



- FullTime = baseSalary + bonus
- PartTime = hoursWorked × hourlyRate
- main: `ArrayList<Employee>` ใส่ ("E001","Anan",30000,5000) และ ("E002","Suda",80,200) วนพิมพ์และรวมเงินเดือน

```
ID: E001 Name: Anan Salary: 35000.0
ID: E002 Name: Suda Salary: 16000.0
Total: 51000.0
```

✍️ เขียนลงกระดาษก่อน แล้วค่อยเปิดหน้าถัดไป

เฉลย ข้อ 1

เฉลยข้อ 1 · ระบบเงินเดือนพนักงาน

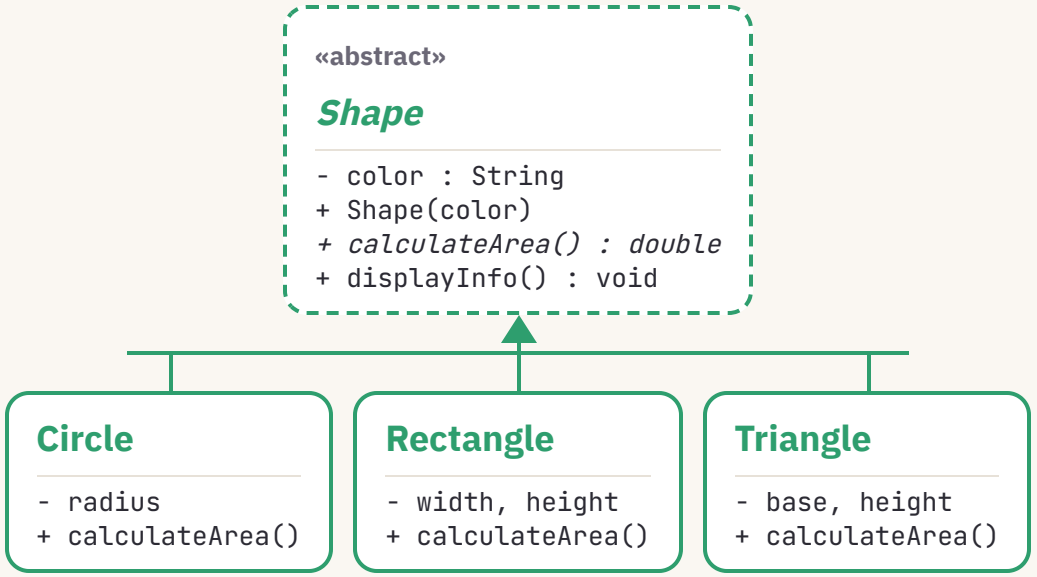
```
import java.util.ArrayList;
abstract class Employee {
    private String employeeId;
    private String name;
    public Employee(String employeeId, String name) {
        this.employeeId = employeeId;
        this.name = name;
    }
    public String getEmployeeId() { return employeeId; }
    public String getName() { return name; }
    public abstract double calculateSalary();
    public void displayInfo() {
        System.out.println("ID: " + employeeId + " Name: " + name + " Salary:
" + calculateSalary());
    }
}
class FullTimeEmployee extends Employee {
    private double baseSalary;
    private double bonus;
    public FullTimeEmployee(String employeeId, String name, double baseSalary,
double bonus) {
        super(employeeId, name);
        this.baseSalary = baseSalary;
        this.bonus = bonus;
    }
    @Override
    public double calculateSalary() { return baseSalary + bonus; }
}
```

```
class PartTimeEmployee extends Employee {
    private int hoursWorked;
    private double hourlyRate;
    public PartTimeEmployee(String employeeId, String name, int hoursWorked,
double hourlyRate) {
        super(employeeId, name);
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }
    @Override
    public double calculateSalary() { return hoursWorked * hourlyRate; }
}
public class Q1 {
    public static void main(String[] args) {
        ArrayList<Employee> list = new ArrayList<>();
        list.add(new FullTimeEmployee("E001", "Anan", 30000, 5000));
        list.add(new PartTimeEmployee("E002", "Suda", 80, 200));
        double total = 0;
        for (Employee e : list) {
            e.displayInfo();
            total += e.calculateSalary();
        }
        System.out.println("Total: " + total);
    }
}
```

จุดให้คะแนน: `abstract` + เมธอดไม่มี body · ลูก `extends` · `super(employeeId, name)` บรรทัดแรก · `@Override` ครบ 2 คลาส

ฝึกข้อเขียน · ข้อ 2 (2.1 UML → โค้ด)

ข้อ 2 · รูปทรงเรขาคณิต



- วงกลม π·r² (`Math.PI`) · สี่เหลี่ยม กว้าง×สูง · สามเหลี่ยม ½×ฐาน×สูง
- `displayInfo()` พิมพ์ Color และ Area ทศนิยม 2 ตำแหน่ง
- main: `Shape[]` = Circle("Red",5), Rectangle("Blue",4,6), Triangle("Green",8,3) รวมพื้นที่

```
Total area: 114.54
```

✍️ เขียนลงกระดาษก่อน แล้วค่อยเปิดหน้าถัดไป

เฉลย ข้อ 2

เฉลยข้อ 2 · รูปทรงเรขาคณิต

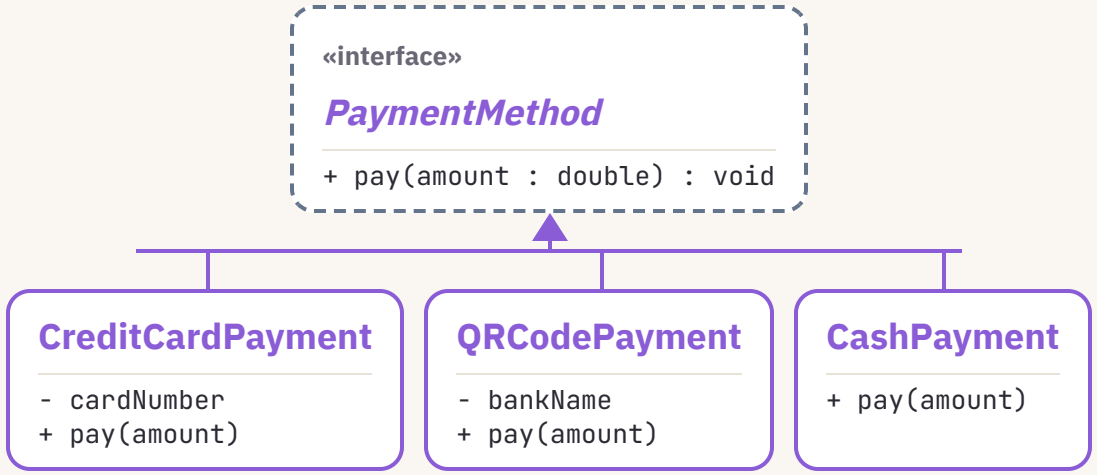
```
abstract class Shape {
    private String color;
    public Shape(String color) { this.color = color; }
    public String getColor() { return color; }
    public abstract double calculateArea();
    public void displayInfo() {
        System.out.println("Color: " + color);
        System.out.printf("Area: %.2f%n", calculateArea());
    }
}
class Circle extends Shape {
    private double radius;
    public Circle(String color, double radius) { super(color); this.radius =
radius; }
    @Override
    public double calculateArea() { return Math.PI * radius * radius; }
}
```

```
class Rectangle extends Shape {
    private double width, height;
    public Rectangle(String color, double width, double height) {
        super(color); this.width = width; this.height = height;
    }
    @Override
    public double calculateArea() { return width * height; }
}
class Triangle extends Shape {
    private double base, height;
    public Triangle(String color, double base, double height) {
        super(color); this.base = base; this.height = height;
    }
    @Override
    public double calculateArea() { return 0.5 * base * height; }
}
public class Q2 {
    public static void main(String[] args) {
        Shape[] shapes = { new Circle("Red", 5.0), new Rectangle("Blue", 4.0,
6.0), new Triangle("Green", 8.0, 3.0) };
        double total = 0;
        for (Shape s : shapes) { s.displayInfo(); total += s.calculateArea();
}
        System.out.printf("Total area: %.2f%n", total);
    }
}
```

จุดให้คะแนน: `new Shape()` ไม่ได้ แต่ `Shape s = new Circle(...)` ได้ · ลูกเรียก `super(color)` · ลูป `for (Shape s : shapes)`

ฝึกข้อเขียน · ข้อ 3 (2.1 UML → โค้ด)

ข้อ 3 · ระบบชำระเงิน



- เส้น**ประ** = `implements`
- เขียน `static void processPayment(PaymentMethod p, double amount)` ที่เรียก `p.pay(amount)`

```
Paid 1500.00 baht by credit card 1234
Paid 750.00 baht by QR Code via ABC Bank
Paid 300.00 baht by cash
```

✍️ เขียนลงกระดาษก่อน แล้วค่อยเปิดหน้าถัดไป

เฉลย ข้อ 3

เฉลยข้อ 3 · ระบบชำระเงิน

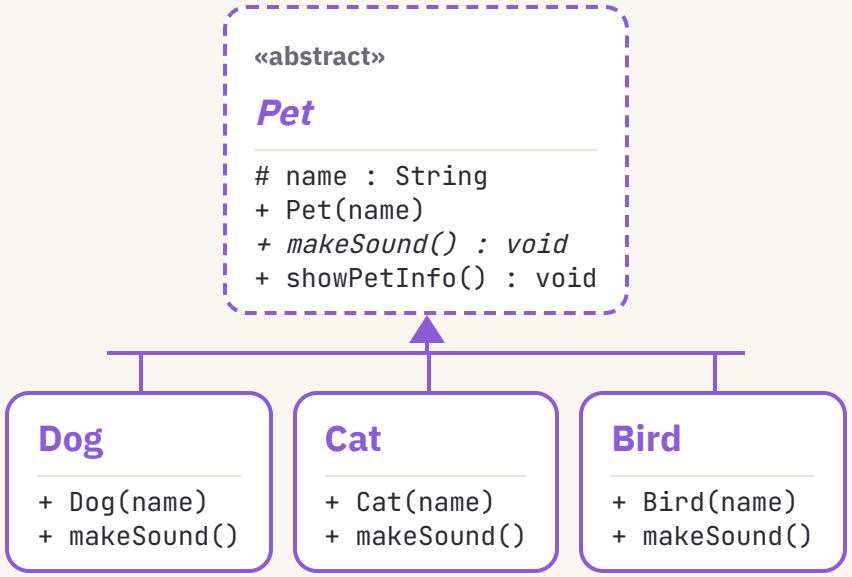
```
interface PaymentMethod {
    void pay(double amount);
}
class CreditCardPayment implements PaymentMethod {
    private String cardNumber;
    public CreditCardPayment(String cardNumber) { this.cardNumber =
cardNumber; }
    @Override
    public void pay(double amount) {
        System.out.printf("Paid %.2f baht by credit card %s%n", amount,
cardNumber);
    }
}
class QRCodePayment implements PaymentMethod {
    private String bankName;
    public QRCodePayment(String bankName) { this.bankName = bankName; }
    @Override
    public void pay(double amount) {
        System.out.printf("Paid %.2f baht by QR Code via %s%n", amount,
bankName);
    }
}
```

```
class CashPayment implements PaymentMethod {
    @Override
    public void pay(double amount) {
        System.out.printf("Paid %.2f baht by cash%n", amount);
    }
}
public class Q3 {
    public static void processPayment(PaymentMethod p, double amount) {
p.pay(amount); }
    public static void main(String[] args) {
        processPayment(new CreditCardPayment("1234"), 1500);
        processPayment(new QRCodePayment("ABC Bank"), 750);
        processPayment(new CashPayment(), 300);
    }
}
```

จุดให้คะแนน: `interface` เมธอดไม่มี body · ใช้ `implements` · เมธอด `pay` ต้องมี **public** · รับพารามิเตอร์เป็นชนิด interface

ฝึกข้อเขียน · ข้อ 4 (2.1 UML → โค้ด)

ข้อ 4 · ระบบสัตว์เลี้ยง (แบบฝึกบท 10)



- `showPetInfo()` พิมพ์ `name + " Making Sound : "` แล้วเรียก `makeSound()`
- Dog → Bok Bok... · Cat → Meaow Meaow... · Bird → Jib Jib...
- main: `ArrayList<Pet>` ใส่ Buddy, Mimi, Tweety

```
Buddy Making Sound : Bok Bok...
Mimi Making Sound : Meaow Meaow...
Tweety Making Sound : Jib Jib...
```

✍️ เขียนลงกระดาษก่อน แล้วค่อยเปิดหน้าถัดไป

เฉลย ข้อ 4

เฉลยข้อ 4 · ระบบสัตว์เลี้ยง (แบบฝึกบท 10)

```
import java.util.ArrayList;
abstract class Pet {
    protected String name;
    public Pet(String name) { this.name = name; }
    public abstract void makeSound();
    public void showPetInfo() {
        System.out.print(name + " Making Sound : ");
        makeSound();
    }
}
class Dog extends Pet {
    public Dog(String name) { super(name); }
    @Override public void makeSound() { System.out.println("Bok Bok..."); }
}
```

```
class Cat extends Pet {
    public Cat(String name) { super(name); }
    @Override public void makeSound() { System.out.println("Meaow Meaow...");
}
}
class Bird extends Pet {
    public Bird(String name) { super(name); }
    @Override public void makeSound() { System.out.println("Jib Jib..."); }
}
public class Q4 {
    public static void main(String[] args) {
        ArrayList<Pet> pets = new ArrayList<>();
        pets.add(new Dog("Buddy"));
        pets.add(new Cat("Mimi"));
        pets.add(new Bird("Tweety"));
        for (Pet p : pets) p.showPetInfo();
    }
}
```

จุดให้คะแนน: `#` = `protected` · `import java.util.ArrayList;` · ลูกเรียก `super(name)` · แม่เรียก `makeSound()` ได้เพราะ polymorphism

ฝึกข้อเขียน · ข้อ 5 (2.2 เติมโค้ด)

ข้อ 5 · เครื่องคิดเลขหาร try · catch 2 ตัว · finally

เติมช่อง (1)–(8) · 10,2 → Result = 5 · 10,0 → Cannot divide by zero · ABC → Please enter a number (ไม่ถามเลขที่ 2) · ทุกกรณีจบด้วย End Program

```
import java.util.Scanner;
import java.util. (1)          ;

public class Q5 {
    public static void main(String[] args) {
        (2)          sc = new Scanner(System.in);
        (3)          {
            System.out.print("Enter number 1: ");
            int a = sc. (4)          ();
            System.out.print("Enter number 2: ");
            int b = sc.nextInt();
            int result = a / b;
            System.out.println("Result = " + result);
        } catch ( (5)          e) {
            System.out.println("Error: Cannot divide by zero");
        } (6)          ( (7)          e) {
            System.out.println("Error: Please enter a number");
        } (8)          {
            sc.close();
        }
        System.out.println("End Program");
    }
}
```

เฉลย ข้อ 5

เฉลยข้อ 5 · เครื่องคิดเลขหาร

ช่อง	คำตอบ
(1)	`InputMismatchException`
(2)	`Scanner`
(3)	`try`
(4)	`nextInt`

ช่อง	คำตอบ
(5)	`ArithmeticException`
(6)	`catch`
(7)	`InputMismatchException`
(8)	`finally`

เมื่อเกิด exception บรรทัดที่เหลือใน try ถูกข้ามทันที จึงไม่ถามเลขที่ 2 · `InputMismatchException` ต้อง import จาก `java.util` · finally ทำเสมอ

ฝึกข้อเขียน · ข้อ 6 (2.2 เติมโค้ด)

ข้อ 6 · บัญชีธนาคาร

throw · throws · custom exception

เติมช่อง (1)–(8) · ผลลัพธ์: Balance: 700.0 → Error: Insufficient balance → Thank you

```
class InsufficientBalanceException  (1)          {
    public InsufficientBalanceException(String message) {
         (2)         (message);
    }
}
class BankAccount {
    private double balance;
    public BankAccount(double balance) {
        if (balance < 0) {
             (3)          IllegalArgumentException("Balance cannot be
negative");
        }
        this.balance = balance;
    }
    public void withdraw(double amount)  (4)
InsufficientBalanceException {
        if (amount > balance) {
             (5)         ("Insufficient balance");
        }
        balance -= amount;
    }
    public double getBalance() { return balance; }
}
public class Q6 {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(1000);
        try {
            acc.withdraw(300);
            System.out.println("Balance: " + acc.getBalance());
            acc.withdraw(5000);
            System.out.println("This line is skipped");
        } catch ( (6)          e) {
            System.out.println("Error: " + e. (7)          ());
        }  (8)          {
            System.out.println("Thank you");
        }
    }
}
```

เฉลย ข้อ 6

เฉลยข้อ 6 · บัญชีธนาคาร

ช่อง	คำตอบ
(1)	`extends Exception`
(2)	`super`
(3)	`throw new`
(4)	`throws`

ช่อง	คำตอบ
(5)	`throw new InsufficientBalanceException`
(6)	`InsufficientBalanceException`
(7)	`getMessage`
(8)	`finally`

throw + `new` = โยนในเนื้อเมธอด · **throws** = ประกาศที่หัวเมธอด · `extends Exception` = checked จึงต้องมี try-catch ตอนเรียก · บรรทัด "This line is skipped" ไม่ถูกพิมพ์

ฝึกข้อเขียน · ข้อ 7 (2.2 เติมโค้ด)

ข้อ 7 · โปรแกรมบวกเลข GUI

Layout · Event · try-catch ใน GUI

เติมช่อง (1)–(16) · ฟอร์ม 2×2 ด้านบน · ผลลัพธ์ตรงกลาง · ปุ่มด้านล่าง · กรอกตัวอักษรต้องเด้ง error

```
import javax.swing.*;
import java.awt. (1)          ;
import java.awt.GridLayout;
public class Q7  (2)          {
    private JTextField firstField, secondField;
    private JLabel resultLabel;
    public Q7() {
        firstField = new JTextField(10);
        secondField = new JTextField(10);
        JButton calculateButton = new JButton("คำนวณ");
        resultLabel = new JLabel("ผลลัพธ์จะแสดงที่นี่", SwingConstants.CENTER);
        JPanel formPanel = new JPanel(new  (3)          (2, 2, 5, 5));
        formPanel.add(new JLabel("จำนวนที่ 1:"));
        formPanel. (4)          (firstField);
        formPanel.add(new JLabel("จำนวนที่ 2:"));
        formPanel.add(secondField);
        setLayout(new BorderLayout(10, 10));
        add(formPanel, BorderLayout. (5)          );
        add(resultLabel, BorderLayout.CENTER);
        add(calculateButton, BorderLayout. (6)          );
        calculateButton. (7)          (e -> calculate());
        setTitle("โปรแกรมบวกเลข");
        setSize(400, 220);
        setDefaultCloseOperation(JFrame. (8)          );
        setLocationRelativeTo(null);
    }
    private void calculate() {
         (9)          {
            double a = Double. (10)          (firstField. (11)
().trim());
            double b = Double.parseDouble(secondField.getText().trim());
            resultLabel. (12)          ("ผลลัพธ์ = " + (a + b));
        } catch ( (13)          e) {
            JOptionPane. (14)          (this, "กรุณากรอกข้อมูลเป็นตัวเลข",
                    "ข้อมูลไม่ถูกต้อง", JOptionPane. (15)          );
        }
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q7(). (16)          (true));
    }
}
```

เฉลย ข้อ 7

เฉลยข้อ 7 · โปรแกรมบวกเลข GUI

ช่อง	คำตอบ
(1)	`BorderLayout`
(2)	`extends JFrame`
(3)	`GridLayout`
(4)	`add`
(5)	`NORTH`
(6)	`SOUTH`
(7)	`addActionListener`
(8)	`EXIT_ON_CLOSE`

ช่อง	คำตอบ
(9)	`try`
(10)	`parseDouble`
(11)	`getText`
(12)	`setText`
(13)	`NumberFormatException`
(14)	`showMessageDialog`
(15)	`ERROR_MESSAGE`
(16)	`setVisible`

`getText()` ได้ String ต้อง `parseDouble` ก่อนบวก · พิมพ์ตัวอักษร → `NumberFormatException` · ไม่มี `addActionListener` = กดปุ่มแล้วเงียบ · `setVisible(true)` ท้ายสุด

ฝึกข้อเขียน · ข้อ 8 (2.2 เติมโค้ด)

ข้อ 8 · BMI Calculator anonymous class · catch 2 ตัว

เติมช่อง (1)–(11) · BMI = kg / m² · <18.5 Under · <25 Normal · <30 Over · ≥30 Obese · 65 kg / 170 cm → 22.49 Normal

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event. (1)          ;
class BMI {
    private double weight, heightCm;
    public BMI(double weight, double heightCm) {
        if (weight <= 0 || heightCm <= 0)  (2)          new
IllegalArgumentException("ค่าต้องมากกว่า 0");
        this.weight = weight; this.heightCm = heightCm;
    }
    public double calculate() { double m = heightCm / 100; return  (3)          ; }
    public String getResult() {
        double b = calculate();
        if (b <  (4)          ) return "Underweight";
        else if (b < 25) return "Normal";
        else if (b < 30) return "Overweight";
        else return "Obese";
    }
}
public class Q8 extends JFrame {
    private JTextField weightField = new JTextField(10), heightField = new
JTextField(10);
    private JLabel resultLabel = new JLabel("BMI: -", SwingConstants.CENTER);
    public Q8() {
        JButton calcButton = new JButton("Calculate");
        JPanel form = new JPanel(new GridLayout(2, 2, 5, 5));
        form.add(new JLabel("Weight (kg):")); form.add(weightField);
        form.add(new JLabel("Height (cm):")); form.add(heightField);
        setLayout(new BorderLayout(10, 10));
        add(form, BorderLayout.NORTH);
        add(resultLabel, BorderLayout.CENTER);
        add(calcButton, BorderLayout.SOUTH);
        calcButton.addActionListener( (5)          {
            @Override
            public void  (6)          ( (7)          e) {
                try {
                    double w = Double.parseDouble(weightField.getText().trim());
                    double h = Double.parseDouble(heightField.getText().trim());
                    BMI bmi =  (8)          ;
                    resultLabel.setText(String.format("BMI: %.2f  Result: %s",
bmi.calculate(), bmi.getResult()));
                } catch ( (9)          ex) {
                    JOptionPane.showMessageDialog(Q8.this, "กรุณากรอกตัวเลข", "Error",
JOptionPane.ERROR_MESSAGE);
                } catch ( (10)          ex) {
                    JOptionPane.showMessageDialog(Q8.this, ex. (11)          (), "Error",
JOptionPane.WARNING_MESSAGE);
                }
            }
        });
        setTitle("BMI Calculator"); setSize(350, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); setLocationRelativeTo(null);
    }
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> new Q8().setVisible(true));
    }
}
```

เฉลย ข้อ 8

เฉลยข้อ 8 · BMI Calculator

ช่อง	คำตอบ
(1)	`ActionListener`
(2)	`throw`
(3)	`weight / (m * m)`
(4)	`18.5`
(5)	`new ActionListener()`
(6)	`actionPerformed`

ช่อง	คำตอบ
(7)	`ActionEvent`
(8)	`new BMI(w, h)`
(9)	`NumberFormatException`
(10)	`IllegalArgumentException`
(11)	`getMessage`

คลาส BMI = Model ไม่รู้จัก GUI · `new ActionListener(){…}` = anonymous class ต้อง override `actionPerformed(ActionEvent e)` · catch `NumberFormatException` ต้องอยู่**ก่อน** `IllegalArgumentException` เพราะเป็นคลาสลูก